

■ kubeaudit — Docker Security Update

Stage 1 Extension: Running Container Security Auditing

This document describes the Docker security scanning extension added to the kubeaudit CLI. The new **kubeaudit docker** command connects to the Docker Engine via its Unix socket, inspects every running container, and reports security misconfigurations across **10 check categories** producing findings with IDs **DOCK-001 through DOCK-012**.

10	12	4	43
Check Categories	Finding IDs	Severity Levels	Total Tests Pass

1. Architecture Overview

The Docker scanner is added as a parallel scan domain alongside Kubernetes. It follows the exact same pipeline pattern: client → fetch → parallel check workers → merge → JSON report.

<code>pkg/docker/types.go</code>	Docker API types (ContainerInspect, HostConfig, MountPoint ...)
<code>pkg/docker/client.go</code>	Unix socket client — connects to <code>/var/run/docker.sock</code>
<code>pkg/docker/checks.go</code>	10 security checks with full logic (DOCK-001 ... DOCK-012)
<code>pkg/docker/scanner.go</code>	Parallel orchestrator — goroutine pool, merge, dedup
<code>pkg/docker/checks_test.go</code>	23 unit tests covering all checks
<code>cmd/docker.go</code>	kubeaudit docker subcommand — flags, scan, JSON output, exit code

Data flow: kubeaudit docker

1	CLI flag parse --socket --pretty --verbose --workers --timeout
2	Docker client Dial <code>/var/run/docker.sock</code> → GET <code>/_ping</code> to verify
3	Resource fetch GET <code>/containers/json?filters=running</code> → inspect each
4	Parallel workers jobs chan DockerCheck → goroutine pool → results chan
5	10 checks run Each <code>check.Run(*ResourceSet)</code> → []Finding
6	Merge + dedup merger goroutine collects all findings, removes duplicates
7	JSON output ScanResult → json.Marshal → stdout
8	CI exit code 0=clean 1=LOW/MED 2=HIGH 3=CRITICAL

2. Docker Security Checks

DOCK-001	Privileged Container	CRITICAL
What it checks:	Detects containers launched with <code>--privileged</code> . A privileged container has full access to all Linux kernel capabilities and host devices — effectively root on the host machine. This is the most dangerous Docker misconfiguration.	
Remediation:	Remove <code>--privileged</code> . Use specific <code>--cap-add</code> entries for only the capabilities the application actually requires.	
DOCK-002	Root User	HIGH
What it checks:	Flags containers whose User is empty, '0', or 'root'. A container process running as UID 0 can exploit kernel bugs or volume misconfigs to escalate to root on the host.	
Remediation:	Add a non-root USER instruction in the Dockerfile. Alternatively, pass <code>--user :</code> at runtime.	
DOCK-003	Dangerous Capabilities	HIGH
What it checks:	Detects containers that add high-risk Linux capabilities via <code>--cap-add</code> : CAP_SYS_ADMIN, CAP_NET_ADMIN, CAP_SYS_PTRACE, CAP_SYS_MODULE, CAP_DAC_OVERRIDE, CAP_SETUID, CAP_SETGID.	
Remediation:	Remove dangerous capabilities from <code>--cap-add</code> . Drop ALL capabilities with <code>--cap-drop=all</code> then add back only what is needed.	
DOCK-004	Writable Root Filesystem	MEDIUM
What it checks:	Flags containers whose root filesystem is not mounted read-only. A writable rootfs allows an attacker to modify binaries, install backdoors, or drop persistence payloads.	
Remediation:	Add <code>--read-only</code> to the container run command. Mount specific writable directories via <code>--tmpfs /tmp</code> or named volumes.	
DOCK-005	Sensitive Host Bind-Mount	CRITICAL/HIGH
What it checks:	Detects bind-mounts of sensitive host paths: <code>/var/run/docker.sock</code> (CRITICAL — gives full Docker daemon control), <code>/proc</code> , <code>/sys</code> , <code>/etc</code> , <code>/root</code> (HIGH), <code>/home</code> , <code>/var/log</code> (MEDIUM).	
Remediation:	Remove the bind-mount. Provide only the specific files the container needs, never entire sensitive directories.	
DOCK-006	Host Network Mode	HIGH
What it checks:	Flags containers using <code>--network=host</code> . The container shares the host network stack, bypassing Docker network isolation. All host ports are directly exposed.	
Remediation:	Use a dedicated Docker bridge or overlay network. Expose only specific ports with <code>-p :</code> .	

DOCK-007	Host PID Namespace	HIGH
What it checks:	Flags containers using <code>--pid=host</code> . The container can see and signal all host processes, including system daemons. This enables process injection and information leakage.	
Remediation:	Remove <code>--pid=host</code> . It is rarely needed outside specific debugging tools.	
DOCK-008	No Memory Limit	MEDIUM
What it checks:	Detects containers with <code>Memory=0</code> (unlimited). An unbounded container can exhaust host memory, triggering the OOM killer and taking down other workloads.	
Remediation:	Set <code>--memory</code> (e.g. <code>--memory=512m</code>) to bound memory consumption.	
DOCK-009	No CPU Limit	LOW
What it checks:	Detects containers with <code>NanoCPUs=0</code> (unlimited). Without CPU limits a container can starve other workloads under heavy load.	
Remediation:	Set <code>--cpus</code> (e.g. <code>--cpus=0.5</code>) to limit CPU usage to a fraction of a core.	
DOCK-010	Secrets in Environment Vars	HIGH
What it checks:	Scans environment variable names for keywords: <code>PASSWORD</code> , <code>SECRET</code> , <code>TOKEN</code> , <code>API_KEY</code> , <code>PRIVATE_KEY</code> , <code>AWS_SECRET</code> , <code>DATABASE_PASSWORD</code> , and more. Secrets in env vars are visible to anyone with <code>docker inspect</code> access.	
Remediation:	Use Docker secrets (<code>docker secret create</code>) or an external secrets manager (HashiCorp Vault, AWS Secrets Manager). Never pass credentials via <code>-e</code> .	
DOCK-011	Unpinned Image Tag	MEDIUM
What it checks:	Flags containers using <code>:latest</code> or images with no tag at all. These cannot guarantee reproducibility and may silently pull a malicious or broken image update.	
Remediation:	Pin to a specific digest: <code>image@sha256:..</code> . Rebuild and redeploy to rotate the image.	
DOCK-012	No-New-Privileges Not Set	MEDIUM
What it checks:	Detects containers missing <code>--security-opt no-new-privileges:true</code> . Without this flag, processes inside the container can gain extra privileges via <code>setuid/setgid</code> binaries in the image.	
Remediation:	Add <code>--security-opt no-new-privileges:true</code> to every container run command.	

3. CLI Usage

Basic Docker scan (all running containers)	<code>kubeaudit docker</code>
Pretty-printed JSON output	<code>kubeaudit docker --pretty</code>
Verbose mode — shows per-check timing	<code>kubeaudit docker --verbose --pretty</code>
Custom socket path (Docker Desktop on macOS)	<code>kubeaudit docker --socket ~/.docker/run/docker.sock</code>
Filter CRITICAL findings with jq	<code>kubeaudit docker jq '.findings[] select(.severity=="CRITICAL")'</code>
Count findings by severity	<code>kubeaudit docker jq '.count_by_severity'</code>
Show only a specific check	<code>kubeaudit docker jq '.findings[] select(.check_name=="docker-privileged")'</code>
Use in CI — fails pipeline on HIGH or CRITICAL	<code>kubeaudit docker; [\$? -le 1] && echo PASS exit 1</code>
Kubernetes scan (unchanged from Stage 1)	<code>kubeaudit scan --namespace production --pretty</code>
Kubernetes scan with context	<code>kubeaudit scan --context staging --verbose</code>

JSON Output Schema

```
{
  "socket_path":      "/var/run/docker.sock",
  "containers_scanned": 3,
  "findings": [
    {
      "id":            "DOCK-001",
      "check_name":    "docker-privileged",
      "category":      "container",
      "severity":       "CRITICAL",
      "resource":       { "kind": "DockerContainer", "name": "redis" },
      "message":        "Container is running in privileged mode ...",
      "remediation":    "Remove --privileged. Use specific --cap-add ...",
      "details":        { "privileged": "true" }
    }
  ],
  "count_by_severity": { "CRITICAL": 1, "HIGH": 2, "MEDIUM": 3, "LOW": 0 },
  "fetch_errors":      []
}
```

4. Test Coverage

Check	Test cases	Status
docker-privileged	Privileged=true flagged, false=clean	PASS
docker-root-user	empty / root / 0 / 0:0 flagged; 1000:1000 clean	PASS
docker-dangerous-caps	CAP_SYS_ADMIN+CAP_NET_ADMIN=2 findings; safe cap=0	PASS
docker-writable-rootfs	ReadOnlyRootfs=false flagged; true=clean	PASS
docker-sensitive-mount	docker.sock=CRITICAL; /etc=HIGH; volume mount ignored	PASS
docker-host-network	NetworkMode=host → DOCK-006; PidMode=host → DOCK-007	PASS
docker-resource-limits	Memory=0 → DOCK-008; NanoCPUs=0 → DOCK-009	PASS
docker-env-secrets	DATABASE_PASSWORD=x flagged; empty value ignored	PASS
docker-image-tag	:latest flagged; no tag flagged; @sha256 digest=clean	PASS
docker-no-new-privileges	Missing opt flagged; opt present=clean	PASS
models (Kubernetes)	Severity string, JSON marshal, ResourceRef, BuildSummary	PASS
orchestrator	merge, dedup, all-run, parallel, cancel, worker-limit	PASS
kubeconfig parser	current-context, clusters, contexts, users, invalid YAML	PASS
JSON reporter	output structure, pretty format, exit codes 0-3	PASS

Total: 43 tests, 0 failures, race detector clean. Run with: `go test -race -count=1 ./...`

Next steps: Stage 2 fills in the Kubernetes check stubs (ContainerCheck, RBACCheck, etc.) with real logic mirroring the Docker checks above. The Docker scanner can then be extended with SBOM-aware CVE scanning (OSV.dev) and integrated into the unified trend report.